ULB MA en STIC Master en sciences et technologies de l'information et de la communication

2025–2026

INFO-H-413 — Heuristic Optimization

Iterative Improvement and Variable Neighborhood Descent for the Linear Ordering Problem

El Mamoune Benmassaoud

Prof. Thomas Stützle

Academic year 2025–2026

Table of Contents

Abstract **1**

1 Introduction **2**

2 Problem Description **3**

2.1 The Linear Ordering Problem . . . 3

2.2 Equivalent Formulations . . . 3

2.3 Applications . . . 3

2.4 Benchmark Instances . . . 4

3 Materials and Methods **5**

3.1 Neighbourhood Structures . . . 5

3.2 Initialisation . . . 5

3.3 Iterative Improvement . . . 6

3.4 Variable Neighbourhood Descent . . . 6

3.5 Experimental Setup . . . 6

4 Results and Discussion **8**

4.1 Overview . . . 8

4.2 Exercise 1.1 — Iterative Improvement . . . 10

4.2.1 Pivoting rule . . . 10

4.2.2 Neighbourhood structure . . . 10

4.2.3 Initialisation . . . 10

4.2.4 Statistical overview . . . 11

4.3 Exercise 1.2 — Variable Neighbourhood Descent . . . 11

4.3.1 VND1 vs VND2 . . . 11

4.3.2 VND vs single-neighbourhood . . . 12

4.3.3 Quality vs. computation time . . . 12

5 Conclusion **13**

Abstract

The Linear Ordering Problem (LOP) is a classical NP-hard combinatorial optimisation problem that consists of finding a permutation of rows and columns of a square matrix such that the sum of superdiagonal entries is maximised. It has applications in various fields including economics, archaeology, and scheduling.

In this work, we implement and evaluate iterative improvement (II) algorithms for the LOP, combining two pivoting rules (first-improvement and best-improvement), three neighbourhood structures (transpose, exchange, and insert), and two initialisation methods (random permutation and the Chenery-Watanabe greedy heuristic), resulting in twelve algorithm configurations. We further implement two Variable Neighbourhood Descent (VND) algorithms that combine all three neighbourhoods in different orderings.

All neighbourhood evaluations use incremental delta computations to avoid redundant objective function recalculations. Experiments are conducted on 78 benchmark instances of sizes 150 and 250. Results are compared using the relative percentage deviation (RPD) from best-known solutions and assessed through paired Wilcoxon signed-rank tests and Student's t-tests.

The results show that insert-based and VND algorithms consistently outperform transpose and exchange neighbourhoods. The initialisation method has a significant impact for transpose and exchange, but not for insert. No statistically significant difference was found between the two VND orderings.

1 Introduction

The Linear Ordering Problem (LOP) is a well-known NP-hard combinatorial optimisation problem with a wide range of practical applications, including the triangularisation of input-output matrices in economics, chronological ordering in archaeology, and scheduling problems [SS04 ; MRD12]. Given the difficulty of finding optimal solutions in reasonable time, heuristic and metaheuristic approaches are commonly used in practice.

Local search methods form the backbone of many successful algorithms for the LOP. They operate by iteratively moving from a current solution to a neighbouring one as long as an improvement can be found, stopping at a local optimum. The quality of the final solution largely depends on the choice of neighbourhood structure and pivoting rule used during the search.

This report describes the implementation and experimental evaluation of iterative improvement (II) algorithms and Variable Neighbourhood Descent (VND) for the LOP. For II, we consider three neighbourhood structures — transpose, exchange, and insert — combined with two pivoting rules (first-improvement and best-improvement) and two initialisation strategies (random permutation and the Chenery-Watanabe greedy heuristic [CW58]), yielding twelve algorithm configurations in total. VND extends this by cycling through multiple neighbourhoods to escape local optima that a single neighbourhood cannot avoid [HM01].

A key aspect of the implementation is the use of incremental delta evaluations for all neighbourhood structures, which avoids recomputing the full objective function at each step and significantly reduces computation time.

The algorithms are tested on 78 benchmark instances of sizes 150 and 250, drawn from established LOP benchmark libraries [MRD12]. Performance is measured using the relative percentage deviation (RPD) from best-known solutions, and statistical comparisons between algorithms are performed using paired Wilcoxon signed-rank tests [Wil45] and Student's t-tests.

The rest of this report is organised as follows. Section 2 formally defines the LOP. Section 3 describes the implemented algorithms and the experimental setup. Section 4 presents and discusses the results. Section 5 concludes.

2 Problem Description

2.1 The Linear Ordering Problem

The Linear Ordering Problem (LOP) is a combinatorial optimisation problem that asks for a permutation of the rows and columns of a square matrix that maximises the sum of superdiagonal entries. It is $\mathcal{NP}$-hard in general, which means no polynomial-time exact algorithm is known for it.

More formally, given an $n \times n$ matrix $C = (c_{ij})$ with non-negative integer entries, the goal is to find a permutation π of $\{1, \ldots, n\}$ that maximises

$$f(\pi) = \sum_{i=1}^{n} \sum_{j=i+1}^{n} c_{\pi_i \pi_j}, \tag{2.1}$$

where π_i is the element placed at position i [SS04]. Summing $c_{\pi_i \pi_j}$ for all $i < j$ collects all matrix entries that end up above the diagonal once rows and columns are reordered according to π.

2.2 Equivalent Formulations

The LOP is also known as the *triangulation problem* : permute the rows and columns of C simultaneously so that the upper triangle is as large as possible (or equivalently, the lower triangle as small as possible). The two formulations are equivalent.

There is also a graph interpretation. Given a complete weighted directed graph on n nodes, a tournament selects one arc from each pair of nodes. An acyclic tournament defines a total ordering of the nodes, and maximising the total arc weight of the compatible arcs is exactly the LOP [MRD12].

Before running any algorithm, each instance is converted to *normal form* : all entries are made non-negative, diagonal entries are set to zero, and for each pair (i, j) the smaller of $\{c_{ij}, c_{ji}\}$ is set to zero. This transformation does not change the optimal permutation but makes comparing algorithm results across instances more meaningful.

2.3 Applications

Despite being a theoretical combinatorial problem, the LOP appears in several practical contexts :

— **Economics :** triangularising input-output tables to reveal the hierarchical structure of production sectors, as originally proposed by Chenery and Watanabe [CW58].

— **Archaeology :** finding the most likely chronological ordering of artefacts found at an excavation site.

— **Scheduling :** ordering tasks or jobs to minimise precedence-constraint violations.

— **Graph drawing :** minimising the number of arc crossings in layered directed graphs.

2.4 Benchmark Instances

The experiments in this report use a set of 78 benchmark instances provided for this exercise, with matrix sizes $n = 150$ and $n = 250$ (49 instances of each size). These instances are large enough to make exact optimisation intractable, which is precisely what motivates using heuristic approaches. For each instance, a best-known objective value is available and is used throughout to compute the relative percentage deviation (RPD), allowing a fair comparison of solution quality across algorithms.

3

Materials and Methods

3.1 Neighbourhood Structures

A neighbourhood $\mathcal{N}(\pi)$ is the set of all permutations reachable from π by applying a single elementary move. Three neighbourhoods were implemented.

Transpose. Swaps two adjacent elements at positions i and $i+1$:

$$\text{transpose}(\pi, i) = (\pi_1, \ldots, \pi_{i-1}, \pi_{i+1}, \pi_i, \pi_{i+2}, \ldots, \pi_n).$$

The neighbourhood has $n-1$ neighbours. The change in objective value reduces to a single matrix lookup :

$$\Delta_T(\pi, i) = c_{\pi_{i+1}\pi_i} - c_{\pi_i\pi_{i+1}}, \tag{3.1}$$

so each evaluation costs $O(1)$ and a full neighbourhood scan costs $O(n)$.

Exchange. Swaps two arbitrary elements at positions $i < j$:

$$\text{exchange}(\pi, i, j) = (\pi_1, \ldots, \pi_j, \ldots, \pi_i, \ldots, \pi_n).$$

The neighbourhood has $n(n-1)/2$ neighbours. The incremental delta is :

$$\Delta_X(\pi, i, j) = (c_{\pi_j\pi_i} - c_{\pi_i\pi_j}) + \sum_{k=i+1}^{j-1} \left[(c_{\pi_j\pi_k} - c_{\pi_k\pi_j}) + (c_{\pi_k\pi_i} - c_{\pi_i\pi_k})\right]. \tag{3.2}$$

Each evaluation costs $O(|i-j|)$, and a full scan costs $O(n^3)$ without further optimisation.

Insert. Removes element π_i from its position and reinserts it at position $j \neq i$. The neighbourhood has $(n-1)^2$ neighbours. As shown in [SS04], any insert move is equivalent to a sequence of $|i-j|$ adjacent transpose moves, which allows incremental evaluation : the delta for inserting π_i at position j is accumulated from the intermediate swap deltas. This brings the cost of a full scan down from $O(n^3)$ to $O(n^2)$.

3.2 Initialisation

Random permutation. A uniformly random permutation is generated using a Fisher-Yates shuffle. It is simple and covers different regions of the search space each time.

Chenery-Watanabe (CW) heuristic. Originally introduced for triangularising economic input-output tables [CW58], this greedy method builds a permutation by repeatedly selecting the element i not yet placed that has the largest row sum :

$$a_i = \sum_{j=1}^{n} c_{ij}. \tag{3.3}$$

The element with the highest a_i is ranked first, and the process continues until all elements are placed. CW runs in $O(n^2)$ and typically produces a much better starting point than a random permutation.

3.3 Iterative Improvement

An iterative improvement algorithm starts from an initial solution and repeatedly applies an improving move until no such move exists. It stops at a local optimum. Two pivoting strategies were used.

First-improvement (FI). The neighbourhood is scanned in a fixed order and the first move with a strictly positive delta is applied immediately. This avoids scanning the full neighbourhood at each step.

Best-improvement (BI). The entire neighbourhood is scanned and the move with the highest delta is applied. Each iteration is more expensive but makes a larger improvement per step.

All six combinations of pivoting rule and neighbourhood were tested with both initialisation strategies, giving $2 \times 3 \times 2 = 12$ configurations.

3.4 Variable Neighbourhood Descent

Variable Neighbourhood Descent (VND) runs a sequence of local searches, each using a different neighbourhood [HM01]. If a search in neighbourhood $\mathcal{N}_k$ finds an improvement, the algorithm restarts from $\mathcal{N}_1$. If no improvement is found, it moves to $\mathcal{N}_{k+1}$. The process stops when no neighbourhood can improve the solution.

Two orderings were tested, both using first-improvement and CW initialisation :

— **VND1 :** Transpose → Exchange → Insert.

— **VND2 :** Transpose → Insert → Exchange.

Transpose is placed first in both cases because it is the cheapest neighbourhood to scan.

3.5 Experimental Setup

All algorithms were implemented in C and compiled with `-O3`. Each configuration was applied once to each of the 78 benchmark instances. Solution quality is measured using the relative percentage deviation (RPD) from the best-known value f^* :

$$\text{RPD} = 100 \times \frac{f^* - f(\pi)}{f^*}. \tag{3.4}$$

An RPD of 0 means the best-known solution was reached ; lower is better. Note that this convention gives non-negative values, since $f(\pi) \leq f^*$ by definition of the best-known solution. Some references use the opposite sign convention, which produces non-positive values, but the interpretation is equivalent.

Statistical comparisons between algorithm pairs use a paired Wilcoxon signed-rank test and a paired t-test at significance level $\alpha = 0.05$, run in R. Both tests are paired because each algorithm is evaluated on the same 78 instances, which removes the effect of instance difficulty. The Wilcoxon test makes no normality assumption ; the t-test is included as a standard complement.

4

Results and Discussion

4.1 Overview

Table 4.1 shows the average RPD and total computation time for all 14 algorithm configurations, sorted from best to worst solution quality.

Table 4.1: Average RPD (%) and total computation time (seconds) across all 78 instances. FI = first-improvement, BI = best-improvement ; Tra/Exc/Ins = transpose, exchange, insert ; Rnd = random init, CW = Chenery-Watanabe.

Algorithm	Pivot	Neighbourhood	Init	Avg RPD (%)	Total time (s)
VND2	FI	Tra→Ins→Exc	CW	1.92	136.1
VND1	FI	Tra→Exc→Ins	CW	2.02	205.7
II	FI	Insert	CW	2.02	138.8
II	FI	Insert	Rnd	2.02	204.2
II	BI	Insert	CW	2.26	35.9
II	BI	Insert	Rnd	2.30	40.2
II	FI	Exchange	Rnd	2.82	228.0
II	FI	Exchange	CW	2.97	189.6
II	BI	Exchange	Rnd	3.60	35.6
II	BI	Exchange	CW	3.85	32.5
II	BI	Transpose	CW	17.08	0.23
II	FI	Transpose	CW	17.09	0.21
II	BI	Transpose	Rnd	34.47	0.002
II	FI	Transpose	Rnd	34.61	0.002

Looking at the table, the neighbourhood choice is clearly the biggest factor. Insert-based algorithms all land around 2–2.3% RPD, exchange around 2.8–3.9%, and transpose is far behind at 17–35%. Figure 4.1 makes this separation very visible.

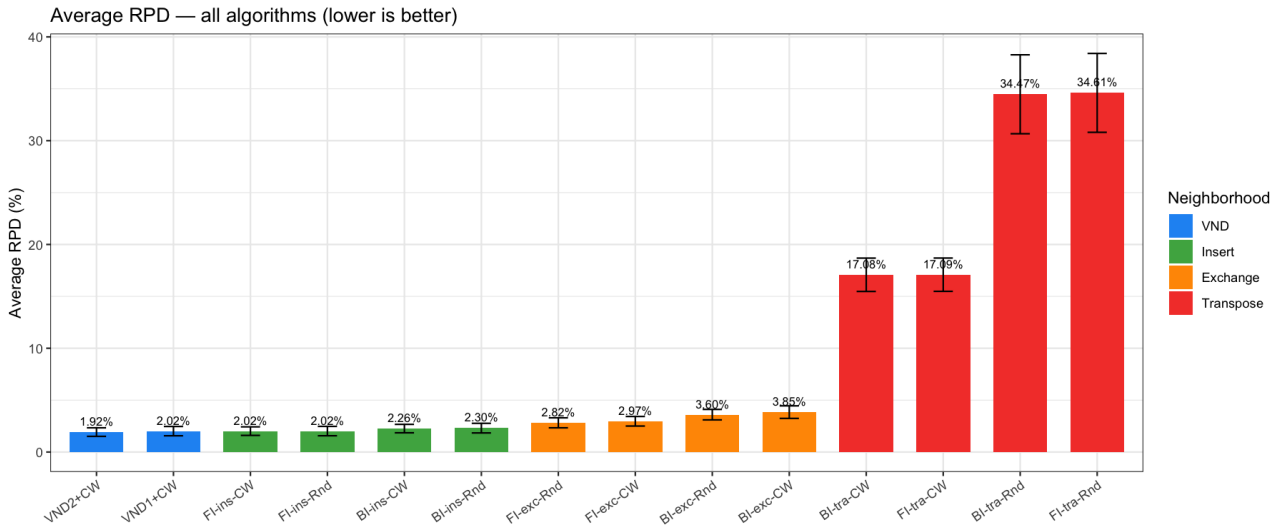

FIGURE 4.1: Average RPD (%) for all 14 configurations. Error bars show 95% confidence intervals. The colour groups (red = Transpose, orange = Exchange, green = Insert, blue = VND) show how much the neighbourhood structure drives the final quality.

The transpose neighbourhood performs so poorly because it only considers $n-1$ adjacent swaps at each step. On instances of size 150 or 250, that is far too few moves to make any real progress, especially from a random starting point. Exchange and insert both consider $O(n^2)$ moves and have much more freedom to improve the solution.

Figure 4.2 shows the full RPD distributions. The insert and VND algorithms are not only better on average but also more consistent across instances. Transpose has a very wide spread, which means it is sensitive to which particular instance it runs on.

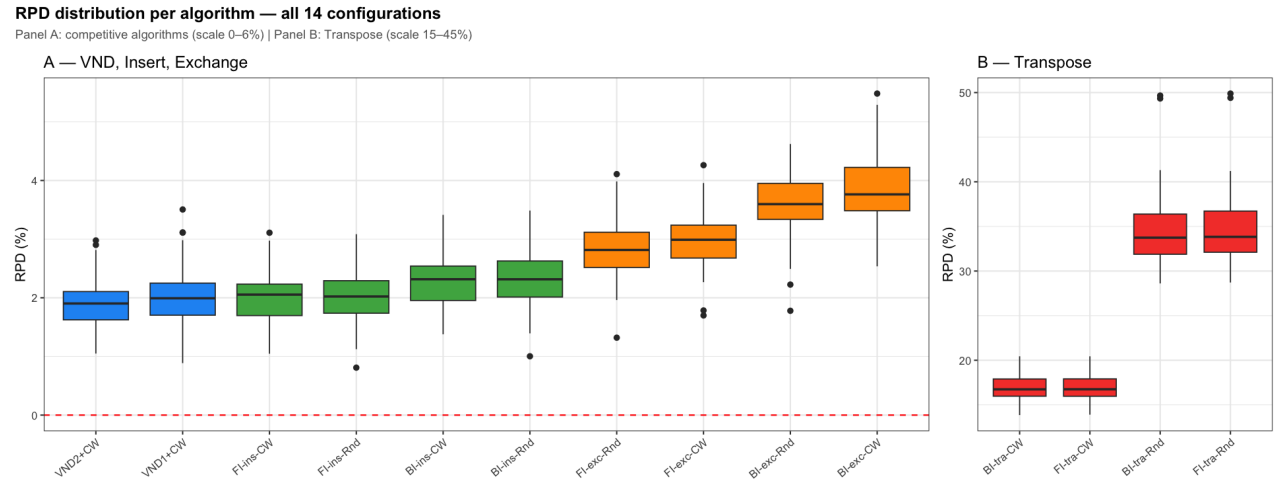

FIGURE 4.2: RPD distribution across the 78 instances. Panel A shows the competitive algorithms (scale 0–6%); Panel B shows transpose on a larger scale (15–45%) to make the spread visible.

4.2 Exercise 1.1 — Iterative Improvement

4.2.1 Pivoting rule

For insert, first-improvement (2.02%) consistently beats best-improvement (2.26–2.30%), and this difference is statistically significant ($p < 0.001$ in all cases). At first this is a bit surprising — you would expect that always picking the best available move leads to a better solution. But what seems to happen is that best-improvement takes fewer, larger steps and gets stuck in local optima that first-improvement avoids by moving more quickly through the search space. A similar pattern holds for exchange.

For transpose, the pivoting rule barely matters once the CW heuristic is used ($p = 0.097$). With only $n - 1$ neighbours to explore, both rules behave almost identically.

4.2.2 Neighbourhood structure

All pairwise comparisons between neighbourhood types are highly significant ($p < 0.001$) : insert beats exchange, and exchange beats transpose, without exception. Figure 4.3 zooms in on the competitive algorithms to make the insert/exchange gap clearer.

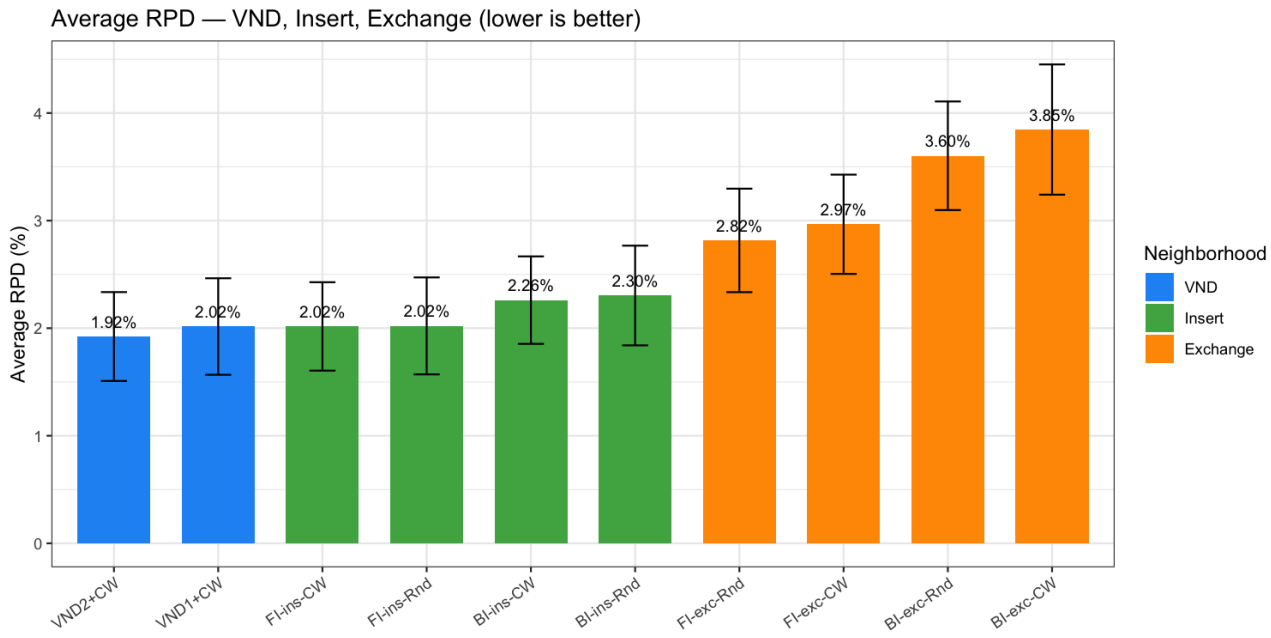

FIGURE 4.3: Average RPD for VND, Insert, and Exchange configurations only (transpose excluded for scale). First-improvement is better than best-improvement in both insert and exchange.

4.2.3 Initialisation

The effect of initialisation depends a lot on the neighbourhood. For transpose, switching from random to CW cuts the RPD from ∼34% down to ∼17% — a massive improvement that is clearly significant ($p < 0.001$). For exchange, CW also helps ($p = 0.014$ for FI, $p = 0.002$ for BI). But for insert, initialisation makes essentially no difference : $p = 0.909$ for FI-insert and $p = 0.557$ for BI-insert. The insert neighbourhood is large enough to fix a bad starting point on its own, so there is not much point in using CW when insert is the chosen neighbourhood.

4.2.4 Statistical overview

Figure 4.4 shows the Wilcoxon p-values for all pairwise comparisons. Almost everything is red (significant), which confirms that most algorithm configurations are genuinely different from one another. The few green cells appear mainly between the best-performing algorithms, where the differences are small.

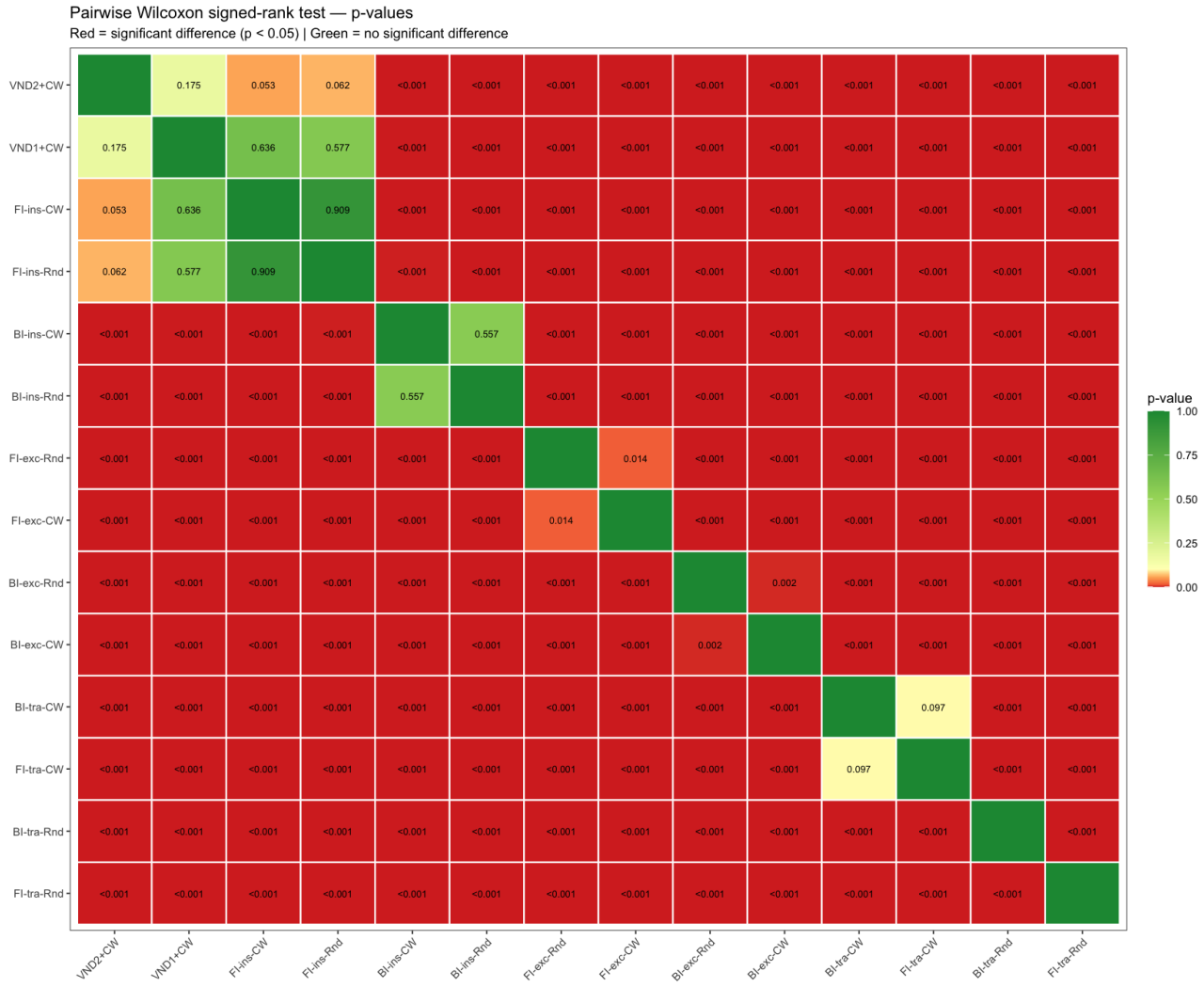

FIGURE 4.4: Pairwise Wilcoxon signed-rank p-values. Red = significant ($p < 0.05$), green = not significant. Most pairs differ significantly ; the exceptions are among the top-ranked algorithms.

4.3 Exercise 1.2 — Variable Neighbourhood Descent

4.3.1 VND1 vs VND2

VND2 (Tra → Ins → Exc) achieves 1.92% RPD and VND1 (Tra → Exc → Ins) achieves 2.02%. The difference is not statistically significant ($p = 0.176$), so we cannot conclude that one ordering is better than the other. In practice, both orderings benefit from the same idea : start with the cheapest neighbourhood (transpose) to pick up easy gains, then switch to the heavier ones to refine the solution.

4.3.2 VND vs single-neighbourhood

Comparing VND to the best single-neighbourhood algorithm (FI-insert-CW, also at 2.02% RPD), the differences are again not significant : $p = 0.636$ for VND1 and $p = 0.053$ for VND2. So adding two extra neighbourhoods on top of insert does not reliably improve the final result on these instances. The insert neighbourhood is already strong enough on its own.

4.3.3 Quality vs. computation time

Figure 4.5 plots average RPD against total computation time. The trade-off is interesting : best-improvement algorithms (for insert and exchange) run much faster but reach worse solutions. First-improvement insert and both VND variants sit in a good spot — competitive quality at moderate cost. Transpose is in a category of its own : extremely fast but nowhere near useful.

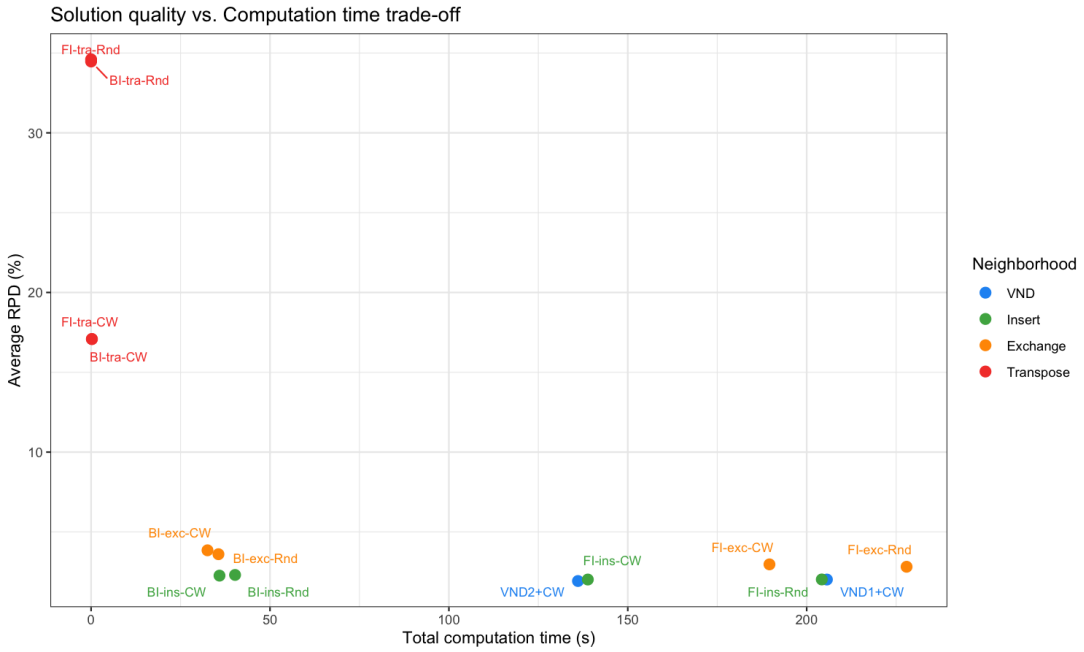

FIGURE 4.5: Average RPD vs total computation time across all 78 instances. Insert FI and VND offer the best quality-to-time trade-off. Transpose is fast but ineffective. Best-improvement runs faster but finds worse local optima than first-improvement.

VND2 is also faster than VND1 (136 s vs 206 s total) while achieving slightly better quality, which suggests that visiting insert before exchange is more efficient — insert likely resolves most of the remaining improvements, leaving less work for exchange to do.

5 Conclusion

This work implemented and tested fourteen algorithm configurations for the Linear Ordering Problem — twelve iterative improvement variants and two VND algorithms — on 78 benchmark instances of sizes 150 and 250.

The most clear result is that the neighbourhood structure matters more than anything else. Insert-based search consistently reaches around 2% average RPD, exchange lands somewhere between 2.8% and 3.9%, and transpose is essentially unusable on its own, reaching 17–35% depending on initialisation. These differences are all statistically significant and persistent across instances, so this is not a fluke of the benchmark set.

The pivoting rule also has a noticeable effect, but in a direction that is not immediately obvious : first-improvement beats best-improvement for both insert and exchange. The most likely explanation is that best-improvement makes fewer, greedier moves and gets trapped in worse local optima, while first-improvement explores the search space more gradually. For transpose, the pivoting rule does not matter much since the neighbourhood is too small either way.

Initialisation is only relevant when using transpose or exchange. For insert, starting from a random permutation or from the CW heuristic gives essentially the same final result ($p > 0.5$). The insert neighbourhood is large enough to recover from a poor starting point, which is a useful practical observation.

The VND results are perhaps the least expected. Combining all three neighbourhoods does not significantly outperform FI-insert-CW on its own ($p = 0.636$ for VND1, $p = 0.053$ for VND2). The insert neighbourhood already covers most of what the others bring, so the added complexity of VND gives limited benefit here. The two VND orderings are also statistically indistinguishable from each other ($p = 0.176$).

If we had to pick one algorithm to use in practice, FI-insert-CW is a solid choice : it matches VND in quality, runs in a reasonable time, and is simpler to implement. BI-insert is an option if speed is more important, though it gives slightly worse results.

One limitation worth mentioning is that each configuration is run only once per instance. For the random initialisation variants, multiple runs would give a more reliable estimate of average performance. A natural follow-up would also be to wrap these local search algorithms inside an iterated local search or simulated annealing framework to escape local optima and push the RPD further down.

Bibliographie

[CW58] Chenery, Hollis B. et Tsunehiko Watanabe (1958). « International Comparisons of the Structure of Production ». In : *Econometrica* 26.4, p. 487-521.

[HM01] Hansen, Pierre et Nenad Mladenović (2001). « Variable Neighbourhood Search : Principles and Applications ». In : *European Journal of Operational Research* 130.3, p. 449-467.

[MRD12] Martí, Rafael, Gerhard Reinelt et Abraham Duarte (2012). « A Benchmark Library and a Comparison of Heuristic Methods for the Linear Ordering Problem ». In : *Computational Optimization and Applications* 51.3, p. 1297-1317.

[SS04] Schiavinotto, Tommaso et Thomas Stützle (2004). « The Linear Ordering Problem : Instances, Search Space Analysis and Algorithms ». In : *Journal of Mathematical Modelling and Algorithms* 3, p. 367-402.

[Wil45] Wilcoxon, Frank (1945). « Individual Comparisons by Ranking Methods ». In : *Biometrics Bulletin* 1.6, p. 80-83.